

Love Letter
Projet AP4B
Version de pré-rendu

Julie BENED
Corentin HAUTEFAYE
Loïc MORIN
Théo RIGAL

19 décembre 2025

Introduction

Dans le cadre de l'UE AP4B (*Semestre A25*), nous avons réalisé une adaptation du jeu *Love Letter* en Java. L'objectif principal de ce projet est de concevoir une application fonctionnelle, robuste, clairement structurée, tout en respectant les principes fondamentaux de la programmation orientée objet.

L'accent est ainsi mis sur la conception UML, l'organisation du code ainsi que la capacité à modéliser efficacement les entités essentielles d'un moteur de jeu simple.

Par ailleurs, cette adaptation vise à préserver l'esprit et les règles du jeu original, tout en y intégrant une dimension créative et ludique en lien avec le contexte universitaire.

Table des matières

1	Présentation	4
1.1	Règles du jeu	4
1.2	Conventions	4
1.3	Outils utilisés	4
2	Réalisation du jeu	5
2.1	Choix	5
2.2	Architecture générale	5
2.3	Organisation des paquets	5
3	Conception UML	6
3.1	Diagramme de classes	7
3.2	Diagramme de cas d'utilisation	8
3.3	Diagramme de séquence	9
4	Retour d'expérience	10
4.1	Organisation	10
4.2	Difficultés rencontrées	10
4.3	Suggestions d'amélioration	10
5	Conclusion	11

1 Présentation

1.1 Règles du jeu

1.2 Conventions

Afin d'assurer une meilleure lisibilité ainsi qu'une cohérence globale du projet, plusieurs conventions ont été adoptées tout au long du développement.

Tout d'abord, le code source et les commentaires sont rédigés en Anglais ; cela est en effet conforme aux standards couramment utilisés en programmation, notamment en entreprise. La typographie utilisée est la norme Java : les noms de classes utilisent la notation *PascalCase*, tandis que les noms de méthodes et attributs suivent le format *camelCase*. Les constantes sont placées dans un fichier dédié (`Const.java`) et sont notées en majuscules avec des *underscores* en tant que séparateurs. En sus de cela, les classes spéciales comme les énumérateurs et interfaces, sont respectivement préfixées par un **E** et un **I**. Enfin, les commentaires suivent le format *Doxygen* ; ainsi la signature de chaque fonction/méthode est donnée en entête du corps.

Les diagrammes UML respectent les conventions classiques de modélisation. Les attributs, méthodes et relations sont clairement identifiées. Notons toutefois que les méthodes triviales, comme les *accesseurs* et *mutateurs*, ne sont pas représentés.

1.3 Outils utilisés

Pour réaliser ce projet, les outils suivants ont été utilisés :

- **IntelliJ IDEA** pour l'environnement de développement
- **Git** et **GitHub** pour le contrôle des versions
- **L^AT_EX** pour la rédaction du rapport
- **tikz-uml** pour dessiner les diagrammes UML en L^AT_EX

Aucune version de Java n'a été spécifiée pour ce sujet. Ainsi, nous avons choisi d'utiliser Java 21. En effet, il s'agit d'une version **LTS** (*Long Term Support*), i.e une version stable pour le développement, et qui reste relativement moderne et sécurisée.

2 Réalisation du jeu

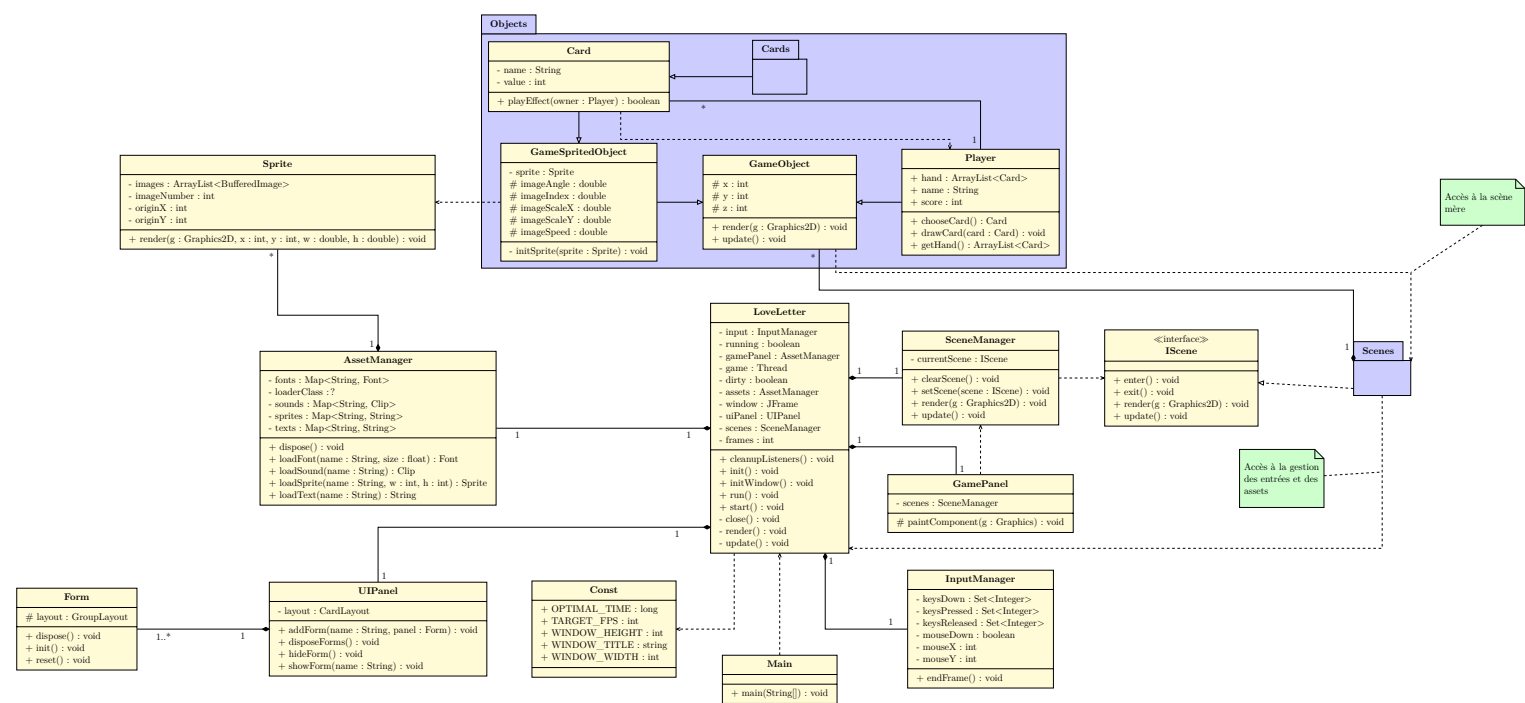
2.1 Choix

2.2 Architecture générale

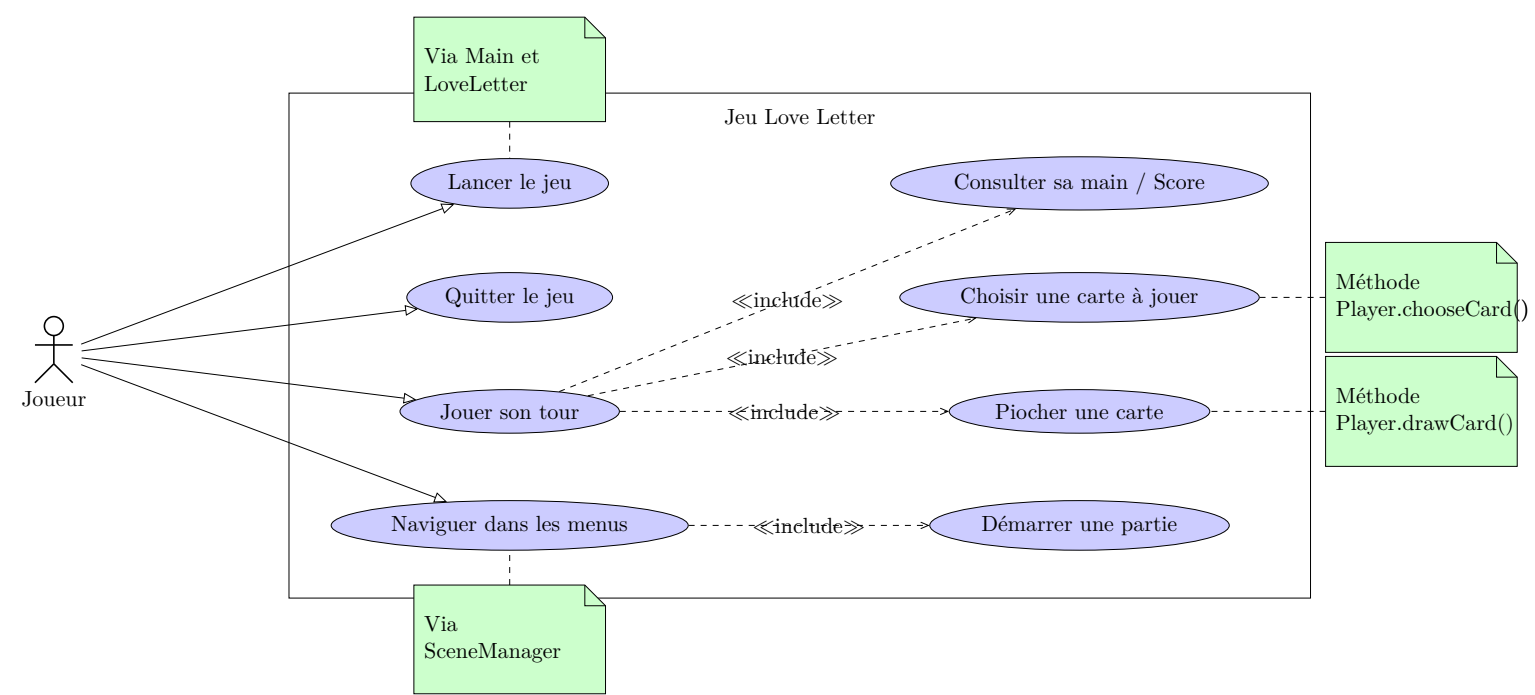
2.3 Organisation des paquets

3 Conception UML

3.1 Diagramme de classes



3.2 Diagramme de cas d'utilisation



3.3 Diagramme de séquence

INSERT HERE

4 Retour d'expérience

4.1 Organisation

4.2 Difficultés rencontrées

4.3 Suggestions d'amélioration

5 Conclusion